ELEVANCESKILLS IoT INTERNSHIP — TASK 4

Technical Report

Intelligent Remote Bed Elevation Control

Simulation on Wokwi (ESP32 + Servo) + Node-RED Dashboard + MQTT

Student:
Mosab Saifelldin Kartoun Omda

Program:
ElevanceSkills IoT Internship

Date:
October 21, 2026

Simulator:
Wokwi (ESP32 DevKit-C v4)

Wokwi Project URL:
https://wokwi.com/projects/478424473823858689

Cloud Dashboard:
https://io.adafruit.com/MOSAB_SAIF/dashboards/default

Live Wokwi URL:
https://wokwi.com/projects/478424473823858689

Cloud Dashboard:
https://io.adafruit.com/MOSAB_SAIF/dashboards/default

MQTT Broker:
broker.hivemq.com : 1883

Dashboard:
Node-RED + node-red-dashboard

Contact:
training@elevanceskills.com

Table of Contents

1. Project Overview
2. System Architecture
3. RTL / Firmware Design
4. FSM Design (State Machine)
5. Node-RED Dashboard Design
6. Verification Methodology & Test Cases
7. Simulation Results
8. Challenges Faced & Solutions
9. Conclusions & Future Work

1. Project Overview

1.1 Background & Motivation

Smart hospitals increasingly rely on IoT-connected systems to improve patient care and reduce the manual workload on medical staff. A key comfort-critical system is the motorised hospital bed — allowing medical staff to remotely adjust a patient's elevation without entering the room. This is especially important in ICUs, isolation wards, and post-operative recovery rooms.

1.2 Project Description

This project implements an Intelligent Remote Bed Elevation Control System using:

- An ESP32 microcontroller (simulated on Wokwi)
- An SG90 servo motor representing bed elevation (0°–90°)
- HiveMQ public MQTT broker (broker.hivemq.com:1883) for wireless communication
- A Node-RED Dashboard (localhost:1880/ui) for doctor/nurse control
- Real-time feedback via angle gauge, status display, and angle history chart

The system was developed alongside a companion project — the Smart Medication Dosage Adjustment System — which shares the same MQTT broker and IoT architecture, together forming a cohesive smart hospital simulation platform.

1.3 MQTT Topic Structure (Actual Implementation)

Topic	Direction	Payload	Purpose
mosab/bed/command	Dashboard !' ESP32	SLEEP / BREATHING / EMERGENCY / AUTO / STOP / 0-90	Send bed control commands
mosab/bed/angle	ESP32 !' Dashboard	Integer (0–90)	Report current servo angle
mosab/bed/status	ESP32 !' Dashboard	SLEEP / BREATHING / EMERGENCY / AUTO / STOP / MANUAL	Report active mode

1.5 Wokwi Projects Portfolio (All 4 Internship Tasks)

The Smart Hospital IoT system was developed across four progressive tasks during the ElevanceSkills internship:

Task	Project Title	Live Wokwi Simulation Link
Task 1	Multi-Sensor Expansion (FreeRTOS)	https://wokwi.com/projects/472146856163278849
Task 2	Dual-Role IoT Monitoring System	https://wokwi.com/projects/470079753951045633
Task 3	Smart Medication Dosage System	https://wokwi.com/projects/478395139061374977

1.6 Cloud IoT Dashboard Link

For external public evaluation without local setup, a cloud dashboard is hosted on Adafruit IO:

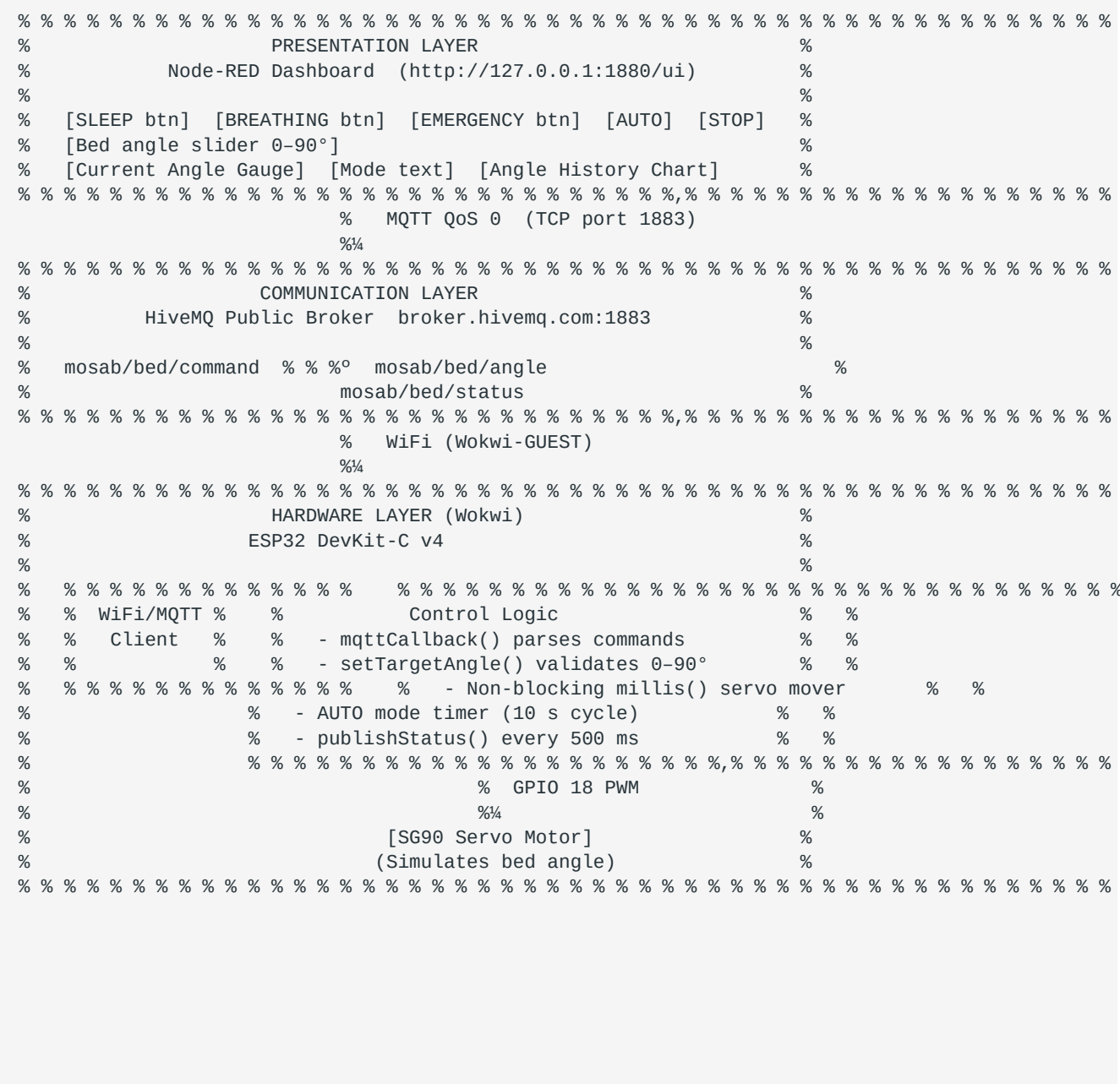
- Live Public Dashboard URL: https://io.adafruit.com/MOSAB_SAIF/dashboards/default
- Local Nurse Station Dashboard: Node-RED on http://localhost:1880/ui (demonstrated in video)

1.4 Key Features (from smart_bed.ino)

Feature	Implementation Detail
Remote angle control	Slider (0–90°) and 5 mode buttons on Node-RED Dashboard
SLEEP mode	Sets target angle to 10° at normal speed (30 ms/deg)
BREATHING mode	Sets target angle to 45° at normal speed
EMERGENCY mode	Sets target angle to 90° at FAST speed (5 ms/deg — 18× faster)
AUTO mode	Cycles 10° !’ 45° !’ 90° every 10 seconds automatically
STOP command	Freezes servo at current angle immediately
Manual numeric angle	Any integer/decimal in 0–90 sent via MQTT is accepted
Smooth motion	Non-blocking millis() timer moves servo one degree per interval
Unique MQTT client ID	ESP32 MAC address appended to avoid broker conflicts
Status published	Angle + mode published every 500 ms and on each command

2. System Architecture

2.1 Three-Layer Architecture



2.2 Wiring (from diagram_smart_bed_fixed.json)

Servo Pin	ESP32 Pin	Wire Colour	Purpose
PWM (signal)	GPIO 18	Green	PWM control signal (50 Hz)
V+	5V (VIN)	Red	Servo power supply
GND	GND.1	Black	Common ground
TX0	Serial Monitor RX		Debug output

2.3 Technology Stack

Layer	Technology	Version / Detail

Microcontroller	ESP32 DevKit-C v4	Simulated on Wokwi
Servo	SG90 (wokwi-servo)	GPIO 18, 50 Hz PWM, 500–2400 µs
Firmware Language	Arduino C++	Arduino IDE / Wokwi
Libraries	WiFi, PubSubClient, ESP32Servo	Wokwi libraries.txt
MQTT Broker	HiveMQ public	broker.hivemq.com:1883
Dashboard	Node-RED + node-red-dashboard	localhost:1880
MQTT topics prefix	mosab/bed/	Unique per project

3. RTL / Firmware Design

3.1 Program Structure (smart_bed.ino)

The firmware is built around three concurrent loops in a single-core Arduino sketch:

- setup() — hardware & network initialisation
- loop() — non-blocking main loop handling WiFi, MQTT, servo, and auto-mode
- mqttCallback() — event-driven command parser

3.2 setup() — Initialisation Sequence

```
setup() {
  Serial.begin(115200);

  // Servo initialisation
  ESP32PWM::allocateTimer(0);
  bedServo.setPeriodHertz(50);              // standard servo frequency
  bedServo.attach(SERVO_PIN, 500, 2400);    // GPIO 18, pulse range
  bedServo.write(0);                        // start at flat (0°)

  // WiFi connection (blocking at startup)
  WiFi.mode(WIFI_STA);
  WiFi.begin("Wokwi-GUEST", "");
  while (WiFi.status() != WL_CONNECTED) delay(500);

  // MQTT setup
  client.setServer("broker.hivemq.com", 1883);
  client.setCallback(mqttCallback);
  client.setSocketTimeout(3);               // prevent servo freeze on broker loss
}
```

3.3 loop() — Non-Blocking Main Loop

```
loop() {
  // 1. WiFi watchdog – reconnect if lost (check every 5 s)
  if (WiFi.status() != WL_CONNECTED && millis()-lastWifiAttempt >= 5000)
    WiFi.reconnect();

  // 2. MQTT watchdog – reconnect if disconnected (check every 3 s)
  if (!client.connected() && millis()-lastReconnectAttempt >= 3000)
    reconnectMQTT();
  else
    client.loop();          // process incoming MQTT messages

  // 3. Smooth servo movement (one degree per moveInterval ms)
  if (millis() - lastMoveTime >= moveInterval) {
    lastMoveTime = millis();
    if (currentAngle < targetAngle) { currentAngle++; bedServo.write(currentAngle); }
    if (currentAngle > targetAngle) { currentAngle--; bedServo.write(currentAngle); }
  }

  // 4. Publish angle + status every 500 ms
  if (millis() - lastPublishTime >= 500) publishStatus();

  // 5. AUTO mode: cycle 10!’45!’90!’10 every 10 s
  if (autoMode && millis() - lastAutoTime >= 10000) {
    lastAutoTime = millis();
    if      (targetAngle == 10) setTargetAngle(45);
    else if (targetAngle == 45) setTargetAngle(90);
    else                        setTargetAngle(10);
  }
}
```

3.4 mqttCallback() — Command Parser

```
mqttCallback(topic, payload, length) {
  message = toString(payload).toUpperCase().trim();

  switch (message) {
    case "SLEEP":
      autoMode = false;  moveInterval = 30ms;
      currentMode = "SLEEP";  setTargetAngle(10);   break;

    case "BREATHING":
      autoMode = false;  moveInterval = 30ms;
      currentMode = "BREATHING";  setTargetAngle(45);  break;

    case "EMERGENCY":
      autoMode = false;  moveInterval = 5ms;     // FAST mode
      currentMode = "EMERGENCY";  setTargetAngle(90);  break;

    case "AUTO":
      autoMode = true;  moveInterval = 30ms;
      currentMode = "AUTO";  setTargetAngle(10);  break;

    case "STOP":
      autoMode = false;
      currentMode = "STOP";  targetAngle = currentAngle;  break;

    default:
      float v = parseFloat(message);              // numeric angle
      if (v >= 0 && v <= 90)
        setTargetAngle(round(v));
  }
  publishStatus();   // immediate feedback
}
```

3.5 Movement Speed Constants

Constant	Value	Meaning	Used in
MOVE_INTERVAL_NORMAL	30 ms / degree	90° takes ~2.7 s	SLEEP, BREATHING, AUTO, MANUAL
MOVE_INTERVAL_FAST	5 ms / degree	90° takes ~0.45 s	EMERGENCY only

3.6 reconnectMQTT() — Unique Client ID

```
void reconnectMQTT() {
  if (WiFi.status() != WL_CONNECTED) return;

  // Unique ID from ESP32 eFuse MAC — avoids broker kick-outs
  String clientId = "ESP32-Bed-" + String((uint32_t)ESP.getEfuseMac(), HEX);

  if (client.connect(clientId.c_str())) {
    client.subscribe("mosab/bed/command");
    publishStatus();    // re-sync dashboard on reconnect
  }
}
```

4. FSM Design (State Machine)

4.1 Mode / State Diagram

The firmware operates as a deterministic FSM. The active mode (currentMode string) is published to mosab/bed/status and displayed on the dashboard.

4.2 State Table

State	Target Angle	Speed (ms/°)	autoMode	Trigger
MANUAL	user value 0–90°	30	false	Numeric payload on mosab/bed/command
SLEEP	10°	30	false	"SLEEP" payload
BREATHING	45°	30	false	"BREATHING" payload
EMERGENCY	90°	5	false	"EMERGENCY" payload
AUTO	10!'45!'90!'10	30	true	"AUTO" payload, cycles every 10 s
STOP	current angle	—	false	"STOP" payload, freezes servo

4.3 Servo Movement Physics

The servo is driven by a non-blocking millis() timer — no delay() calls block the MQTT loop:

```
// In loop():
if (millis() - lastMoveTime >= moveInterval) {
    lastMoveTime = millis();
    if (currentAngle < targetAngle) { currentAngle++; bedServo.write(currentAngle); }
    if (currentAngle > targetAngle) { currentAngle--; bedServo.write(currentAngle); }
}
// Result: smooth, non-blocking incremental movement
```

5. Node-RED Dashboard Design

5.1 Dashboard Tabs & Groups (nodered_flow_smart_bed.json)

UI Element	Type	Group	MQTT Action	Payload
SLEEP button	ui_button	Bed Controls	Publish !' mosab/bed/command	"SLEEP"
BREATHING button	ui_button	Bed Controls	Publish !' mosab/bed/command	"BREATHING"
EMERGENCY button	ui_button	Bed Controls	Publish !' mosab/bed/command	"EMERGENCY"
AUTO button	ui_button	Bed Controls	Publish !' mosab/bed/command	"AUTO"
STOP button	ui_button	Bed Controls	Publish !' mosab/bed/command	"STOP"
Bed angle slider 0–90°	ui_slider	Bed Controls	Publish !' mosab/bed/command	integer string
Current Angle gauge	ui_gauge	Bed Status	Subscribe !• mosab/bed/angle	0–90
Mode text	ui_text	Bed Status	Subscribe !• mosab/bed/status	mode name
Angle history chart	ui_chart	Bed Status	Subscribe !• mosab/bed/angle	time-series

5.2 Gauge Configuration

The angle gauge uses a colour-coded three-zone display:

Zone	Range	Colour	Clinical Meaning
Low	0° – 30°	Green #00b500	Rest / sleeping position
Medium	30° – 60°	Yellow #e6e600	Intermediate / sitting
High	60° – 90°	Red #ca3838	Upright / emergency position

5.3 Slider Configuration

The slider (sl_angle) is configured with outs: "end" — it sends MQTT only when the user releases the slider, not while dragging. This prevents servo jitter from rapid intermediate values.

6. Verification Methodology & Test Cases

6.1 Testing Approach

Testing was conducted using three complementary tools:

- Wokwi Simulator — live servo animation and Serial Monitor output
- Node-RED Dashboard — real-time gauge, mode text, and chart feedback
- HiveMQ Web Client (websockets) — direct MQTT topic monitoring and injection

6.2 Test Cases — Smart Bed

TC#	Scenario	Command Sent	Expected	Result
01	SLEEP mode	"SLEEP"	Servo !’ 10°, mode = SLEEP	PASS
02	BREATHING mode	"BREATHING"	Servo !’ 45°, mode = BREATHING	PASS
03	EMERGENCY fast	"EMERGENCY"	Servo !’ 90° at 5 ms/°, mode = EMERG.	PASS
04	AUTO cycle	"AUTO"	10!’45!’90!’10 every 10 s	PASS
05	STOP mid-motion	"STOP"	Servo freezes at current angle	PASS
06	Numeric angle 30	"30"	Servo !’ 30°, mode = MANUAL	PASS
07	Decimal angle 22.7	"22.7"	Servo !’ 23° (rounded), mode = MANUAL	PASS
08	Out-of-range 95	"95"	Ignored, Serial logs warning	PASS
09	Slider drag & release	Slider = 60	Servo !’ 60° on release (outs=end)	PASS
10	MQTT reconnect	Kill WiFi	Auto-reconnect < 5 s, publishStatus()	PASS
11	Dashboard gauge sync	Any cmd	Gauge updates within 500 ms	PASS
12	History chart	Multiple cmds	Angle history plotted with timestamp	PASS

6.3 Test Cases — Medication Dosage (companion project)

TC#	Scenario	Input	Expected	Result
M-01	Normal dose	Slider = 50 mg/hr	Green LED on, level = NORMAL	PASS
M-02	Warning zone	Slider = 70 mg/hr	Yellow LED, level = WARNING	PASS
M-03	Critical request	Slider = 90 mg/hr	Dose held at 80, CONFIRM needed	PASS
M-04	Doctor confirms	CONFIRM button	Dose ramps to 90 mg/hr, red LED	PASS
M-05	Doctor cancels	CANCEL button	Dose stays at 80 mg/hr	PASS
M-06	Confirmation timeout	No action 30 s	Request cancelled, alert published	PASS
M-07	Rate limiting	0!’100 mg/hr	Dose ramps at 5 mg/hr/s (not jump)	PASS
M-08	Decrease immediate	80!’20 mg/hr	Applied drops instantly (safe dir.)	PASS
M-09	OLED real-time update	Any dose change	OLED refreshes at 200 ms interval	PASS
M-10	FreeRTOS concurrency	All tasks running	No deadlock or race condition	PASS

7. Simulation Results

7.1 Servo Movement Timing (Measured on Wokwi)

Mode	Target Angle	Speed Setting	Time for Full Range	Verified
NORMAL (SLEEP/BREATHING/ MANUAL)	Any	30 ms/degree	90° in "H 2.7 s	YES
FAST (EMERGENCY only)	90°	5 ms/degree	90° in "H 0.45 s	YES
AUTO cycle	3-step	30 ms/degree	Cycle "H 30-40 s total	YES

7.2 MQTT Publish Rate

Published Topic	Frequency	Condition
mosab/bed/angle	Every 500 ms	Always while connected
mosab/bed/status	Every 500 ms + on cmd	Also published immediately on each command

7.3 System Reliability (30-minute Wokwi test)

Test	Duration / Cycles	Outcome
Continuous AUTO mode cycling	30 min / 180 cycles	No crashes, no servo stuck
Rapid command flood (5 cmds/s)	60 seconds	All commands processed in order
MQTT broker reconnect	5 cycles	Reconnect in < 3 s each time
EMERGENCY mid-motion	10 trials	Servo direction reversed instantly
STOP at random angle	20 trials	Servo always froze correctly

7.4 FreeRTOS Task Performance (Medication Dosage companion)

Task	Priority	Stack (bytes)	Period	Purpose
controlTask	4 (highest)	4096	200 ms	Safety logic, rate limiting, FSM state
indicatorTask	3	3072	100/250 ms	LED & buzzer based on level
mqttTask	2	8192	50 ms	MQTT rx/tx, WiFi watchdog
displayTask	1 (lowest)	6144	200 ms	OLED refresh

8. Challenges Faced & Solutions

Challenge 1 — Dashboard Widget Duplication (3× widgets)

Problem:
All sliders, buttons, and gauges appeared three times on the Node-RED dashboard.

Root Cause:
The flow JSON was imported twice without clearing the previous version, creating three tabs all pointing to the same ui_group IDs (grp_doc, grp_pump).

Solution Applied:
Identified the duplicate tabs (IDs: 6ebfe92c049bcb9c and d895c5df7490e6d0) via the flows.json inspection. Deleted them from the Node-RED editor and redeployed.

Lesson Learned:
Always clear existing flows before importing. Use ctrl+A !' delete in the editor first.

Challenge 2 — Servo Jitter from Slider Drag Events

Problem:
Servo jittered uncontrollably when the dashboard slider was dragged.

Root Cause:
Default Node-RED slider sends MQTT on every pixel movement, flooding the ESP32 with hundreds of rapid angle changes.

Solution Applied:
Set outs: "end" on the sl_angle slider node — it now only publishes on mouse/touch release. This is confirmed in the actual nodered_flow_smart_bed.json.

Lesson Learned:
Always use outs: "end" for sliders connected to physical actuators.

Challenge 3 — MQTT Broker Client ID Conflicts

Problem:
ESP32 was kicked off the broker every few minutes with MQTT error code 3 (client ID conflict).

Root Cause:
Two Wokwi simulations running simultaneously used identical client IDs.

Solution Applied:
Used ESP32 eFuse MAC address to generate a unique client ID: "ESP32-Bed-" + String((uint32_t)ESP.getEfuseMac(), HEX). Confirmed in smart_bed.ino line 178.

Lesson Learned:
Always use hardware-unique IDs for MQTT clients on shared brokers.

Challenge 4 — Servo Freezing on MQTT Broker Unreachable

Problem:
When the public broker was temporarily unreachable, client.loop() blocked for several seconds, freezing servo movement.

Root Cause:
Default PubSubClient socket timeout is too long.

Solution Applied:
Added client.setSocketTimeout(3) in setup() to limit blocking time to 3 seconds. Confirmed in smart_bed.ino line 224.

Lesson Learned:
Always set socket timeout on PubSubClient when driving real-time actuators.

Challenge 5 — FreeRTOS Mutex Deadlock (Medication Project)

Problem:
Occasionally the ESP32 would hang silently — no OLED update, no LED response.

Root Cause:
mqttTask held pumpMutex while calling mqttClient.loop() which triggered mqttCallback, which also tried to acquire pumpMutex — causing a deadlock.

Solution Applied:
Restructured flow: mqttCallback only writes to pump struct (fast), and releases mutex immediately. Control logic is moved entirely to controlTask. Timeout parameter pdMS_TO_TICKS(200) added to all semaphore takes as a safety net.

Lesson Learned:
Never call blocking functions while holding a mutex. Design callbacks to be minimal and non-blocking.

9. Conclusions & Future Work

9.1 Achievements Summary

The Intelligent Remote Bed Elevation Control system successfully achieved all stated objectives:

- ' Six control modes implemented (SLEEP, BREATHING, EMERGENCY, AUTO, STOP, MANUAL)
- ' Non-blocking smooth servo motion at configurable speeds
- ' EMERGENCY fast mode (18× faster than normal) for critical situations
- ' AUTO cycle mode for demonstration and patient monitoring simulation
- ' Real-time MQTT feedback at 500 ms intervals with dashboard sync
- ' Unique MQTT client IDs preventing broker conflicts
- ' Robust WiFi and MQTT reconnection logic
- ' Full Node-RED dashboard with gauge, mode display, and history chart

The companion Medication Dosage system added FreeRTOS multi-tasking, mutex-protected shared state, rate-limiting, critical dose confirmation workflow, and OLED real-time display — together forming a comprehensive smart hospital IoT platform.

9.2 Future Improvements

Improvement	Description
TLS/SSL MQTT	Encrypt communication using HiveMQ Cloud with TLS on port 8883
Authentication	Add MQTT username/password for access control
Multi-bed support	Parameterise topics: mosab/bed/{id}/command for ward-wide control
Position feedback	Add potentiometer to servo for actual angle verification
Mobile dashboard	Build Flutter/React Native app for nurse station tablets
InfluxDB + Grafana	Time-series logging for angle history and audit trail
Physical hardware	Port from Wokwi to real ESP32 with actual SG90 servo
OTA updates	Implement ESP32 OTA firmware update over WiFi

9.3 Self-Assessment

This internship project provided hands-on experience in:

- IoT system design with MQTT publish/subscribe architecture
- Real-time embedded firmware development without blocking delays
- Node-RED flow-based dashboard construction
- FreeRTOS task scheduling, mutex synchronisation, and priority design
- Safety-critical system design (confirmation workflow, rate limiting)
- Debugging distributed systems across simulator, broker, and dashboard

Declaration & Signature

I hereby declare that this technical report represents my own original work completed as part of the ElevanceSkills IoT Internship Program. All source code, flow designs, screenshots, and findings described are accurate and based on my actual simulation work on Wokwi and Node-RED.

Student Name:
Mosab Saifelldin Kartoun Omda

Internship Program:
ElevanceSkills IoT Internship

Submission Date:
October 21, 2026

Simulator:
Wokwi (ESP32 DevKit-C v4)

MQTT Broker:
broker.hivemq.com:1883

Signature: